

Pokémon Chronicles: Pocket Beasts

Documentación del Proyecto



Autor: [Marciuc Dragos Stefan]

Índice

1. Descripción del Proyecto	3
1.1. Concepto y Jugabilidad	3
1.2. Características principales	3
1.3. Entorno y Diseño	3
1.4. Mecánicas de Juego	3
1.5. Sistema de Progresión	3
1.6. Detalles Técnicos	4
2. Justificación del Proyecto	4
2.1. Interés Tecnológico	4
2.2. Selección del Género	4
2.3. Valor Formativo	4
3. Alcance del Proyecto	4
4. Alternativas Existentes	5
4.1. Pokémon Oficiales (Nintendo)	5
4.2. Pokémon Showdown	5
4.3. Fangames Varios	5
5. Stack Tecnológico	5
6. Objetivos	6
6.1. Objetivos Principales	6
6.2. Objetivos Secundarios	6
7. Desarrollo del Videojuego	6
7.1. Forma y Mecánica	6
7.2. Mecánicas Principales	7
7.3. Target	7
8. Requisitos del Sistema	7
8.1. Requisitos Funcionales	7
8.2. Requisitos No Funcionales	8
8.3. Requisitos de Interfaz	8
9. Casos de Uso	8
9.1. CU-01: Iniciar Combate	8
9.2. CU-02: Capturar Criatura	9
9.3. CU-03: Gestionar Equipo	9
10. Modelo y Diseño de la Base de Datos	10
10.1. Esquema SQLite	10
10.2. Modelo de Datos en C#	11
10.3. Relaciones Principales	11

11. Guía de Estilos	12
11.1. Logo	13
11.2. Paleta de Colores	13
11.3. Tipografías	14
11.4. Componentes UI	14
12. Mockups de Interfaz	15
12.1. Pantalla de Login	15
12.2. Pantalla de Login del Usuario	16
12.3. Pantalla de configuración del Usuario	17
12.4. Pantalla de login del admin	18
12.5. Pantalla del panel del admin	19
12.6. Pantalla del administrar usuarios	20
12.7. Pantalla de los datos de la partida	20
12.8. Pantalla de configuración del Admin	21
12.9. Pantalla de Flujos	22
12.10 Pantalla Principal del Usuario	22
12.11 Pantalla del Administrador	22
13. Flujo de Navegación	23
13.1. Mapa de Navegación	23
13.2. Descripción del Flujo	23
14. Manual de Usuario	23
14.1. Pantalla de Inicio de Sesión	23
14.1.1. Mensajes de Error	24
14.2. Pantalla de Registro de Nuevo Usuario	24
14.2.1. Validaciones de Registro	25
14.3. Acceso al Mundo del Juego	26
15. Pruebas y Validación del Sistema	26
15.1. Pantallas Implementadas y Estado	26
15.2. Casos de Prueba Ejecutados	27
16. Despliegue y Configuración del Backend	27
16.1. Entorno Local de Desarrollo	27
16.2. Ejecución del Servidor	27
16.3. Arquitectura del Backend	28
16.4. Preparación para Despliegue en Proxmox	29
16.4.1. Plan de Migración a Producción	29
16.5. Consideraciones de Seguridad	29

1 Descripción del Proyecto

1.1 Concepto y Jugabilidad

“Pokémon Chronicles: Pocket Beasts” es un videojuego desarrollado en Unity que recrea las mecánicas clásicas de la saga Pokémon. El jugador podrá explorar un mundo, capturar criaturas y participar en combates por turnos, implementando un RPG funcional con sistemas completos de progresión, combate y gestión de datos.

1.2 Características principales

- **Exploración del mundo:** Movimiento libre por diferentes zonas con encuentros aleatorios
- **Sistema de combate:** Combates por turnos con IA básica enemiga
- **Captura de criaturas:** Sistema de captura con diferentes probabilidades de éxito
- **Progresión:** Sistema de experiencia, niveles y evolución de criaturas
- **Base de datos:** Gestión de datos de criaturas, movimientos y tipos
- **Persistencia:** Guardado y carga de partidas mediante SQLite

1.3 Entorno y Diseño

El juego presenta las siguientes características:

- **Estilo visual:** Gráficos 2D con perspectiva cenital en estilo pixel art retro
- **Interfaz:** HUD de combate e inventario intuitiva y clara
- **Sonido:** Efectos de audio para acciones principales y música ambiental

1.4 Mecánicas de Juego

El sistema de juego está construido sobre Unity, ofreciendo:

Acción	Implementación
Movimiento	Teclas WASD o flechas direccionales
Interacción	Tecla E o clic
Menú	Tecla ESC o botón específico
Combate	Sistema por turnos

Cuadro 1: Controles básicos del juego

1.5 Sistema de Progresión

El juego incluye un sistema de progresión basado en:

- **Experiencia:** Ganada tras ganar combates
- **Niveles:** Subida de nivel con mejora de estadísticas
- **Evoluciones:** Transformación de criaturas al alcanzar niveles específicos

1.6 Detalles Técnicos

El proyecto utiliza:

- **Motor:** Unity 2022.3 LTS
- **Lenguaje:** C#
- **Base de datos:** SQLite para persistencia local
- **Control de versiones:** Git + GitHub

2 Justificación del Proyecto

2.1 Interés Tecnológico

El desarrollo de videojuegos es un campo en constante evolución que combina programación, diseño y creatividad. Este proyecto permite aplicar conocimientos de desarrollo de software en un contexto práctico y atractivo.

2.2 Selección del Género

¿Y por qué un RPG al estilo Pokémon?

- **Compatibilidad técnica:** Género bien documentado con mecánicas establecidas
- **Curva de aprendizaje:** Permite implementar sistemas progresivamente
- **Valor educativo:** Abarca múltiples áreas del desarrollo de software

2.3 Valor Formativo

Este proyecto ofrece:

- Desarrollo de habilidades en C# y Unity
- Implementación de arquitecturas de software
- Gestión de bases de datos y persistencia
- Control de versiones y metodologías ágiles

3 Alcance del Proyecto

El juego incluirá:

- Un mundo explorable con diferentes zonas
- Sistema de combate por turnos funcional
- En un principio 8-10 criaturas únicas con sus evoluciones
- Sistema de captura y gestión de equipo

- Persistencia de datos con SQLite
- Interfaz de usuario completa

Nota: Los elementos finales podrán ajustarse durante el desarrollo según pruebas técnicas.

4 Alternativas Existentes

4.1 Pokémon Oficiales (Nintendo)

- **Ventajas:** Juegos completos y pulidos
- **Desventajas:** Código cerrado, no permiten modificación
- **Diferenciación:** Nuestro proyecto es educativo y de código abierto

4.2 Pokémon Showdown

- **Ventajas:** Sistema de combate muy desarrollado
- **Desventajas:** Solo combates, sin exploración
- **Diferenciación:** Se ofrece experiencia RPG completa

4.3 Fangames Varios

- **Ventajas:** Gran variedad de ideas y enfoques
- **Desventajas:** Calidad variable, pocos documentados
- **Diferenciación:** Enfoque en arquitectura técnica y documentación

5 Stack Tecnológico

Tecnología	Uso	Versión
Unity	Motor principal del juego	2022.3 LTS
C#	Lenguaje de programación	.NET 4.x
SQLite	Base de datos relacional	3.x
Proxmox VE	Virtualización y contenedores	8.x
LXC Container	Contenedor Linux para servicios	Ubuntu 22.04
Git + GitHub	Control de versiones	Latest
Aseprite o variantes	Creación de sprites	Latest

Cuadro 2: Stack tecnológico del proyecto

6 Objetivos

6.1 Objetivos Principales

Los objetivos fundamentales de este proyecto son:

- **Desarrollo del juego funcional:** Crear un RPG que permita:
 - Exploración del mundo con movimiento libre
 - Sistema de combate por turnos funcional
 - Captura y gestión de criaturas
- **Sistema de base de datos:** Implementar persistencia que incluya:
 - Datos de criaturas y movimientos
 - Guardado de partidas de usuario
 - Gestión de progreso del jugador
- **Interfaz de usuario:** Desarrollar interfaces que ofrezcan:
 - HUD de combate clara e intuitiva
 - Menús de gestión de equipo
 - Sistema de navegación fluido

6.2 Objetivos Secundarios

Como metas complementarias se plantean:

- Implementar sistema de evoluciones
- Crear IA básica para combates
- Desarrollar efectos visuales y sonoros

Nota: Estos objetivos podrán ajustarse durante el desarrollo en función de los resultados obtenidos en las primeras pruebas técnicas.

7 Desarrollo del Videojuego

7.1 Forma y Mecánica

- **Género:** RPG por turnos
- **Perspectiva:** 2D cenital
- **Estilo visual:** Pixel art retro
- **Estructura:** Mundo abierto con progresión guiada

7.2 Mecánicas Principales

- **Exploración:**
 - Movimiento libre por el mundo
 - Encuentros aleatorios en hierba
 - Interacción con NPCs y objetos
- **Combate:**
 - Sistema por turnos basado en stats
 - Tipos elementales con ventajas/desventajas
 - 4 movimientos por criatura
- **Progresión:**
 - Sistema de experiencia y niveles
 - Evoluciones en niveles específicos
 - Mejora de stats por nivel

7.3 Target

- Jugadores de 12-35 años
- Fanáticos de RPGs y juegos retro
- Sesiones de 15-45 minutos

8 Requisitos del Sistema

8.1 Requisitos Funcionales

- **RF-01:** Sistema de exploración del mundo
 - Movimiento del jugador por el mapa
 - Encuentros aleatorios con criaturas
 - Transición entre zonas
- **RF-02:** Sistema de combate por turnos
 - Selección de movimientos
 - Cálculo de daño y efectos
 - IA básica para oponentes
- **RF-03:** Captura y gestión de criaturas
 - Sistema de captura con probabilidades
 - Gestión de equipo (6 criaturas máximo)
 - Almacenamiento en PC

- **RF-04:** Progresión y evoluciones
 - Sistema de experiencia y niveles
 - Evolución en niveles específicos
 - Mejora de estadísticas
- **RF-05:** Persistencia de datos
 - Guardado de partidas en SQLite
 - Carga de partidas guardadas
 - Gestión de múltiples usuarios

8.2 Requisitos No Funcionales

- **RNF-01:** Rendimiento
 - 30 FPS en hardware estándar
 - Tiempos de carga menores a 3 segundos
- **RNF-02:** Usabilidad
 - Interfaz intuitiva y clara
 - Curva de aprendizaje suave
 - Feedback visual inmediato
- **RNF-03:** Compatibilidad
 - Windows 10/11
 - Resoluciones 1080p y 720p

8.3 Requisitos de Interfaz

- **RI-01:** HUD de combate
 - Información clara de HP y estado
 - Selección intuitiva de movimientos
 - Indicadores de turno
- **RI-02:** Menús del juego
 - Navegación con teclado/ratón
 - Diseño consistente en todas las pantallas
 - Acceso rápido a funciones principales

9 Casos de Uso

9.1 CU-01: Iniciar Combate

Actor Principal: Jugador

Precondiciones:

- Jugador explorando el mundo
- En zona de hierba o encuentro forzado

Flujo Principal:

1. Sistema detecta encuentro aleatorio
2. Transición a escena de combate
3. Mostrar HUD de combate con opciones
4. Iniciar sistema de turnos
5. Jugador selecciona movimiento
6. Resolver turno y mostrar resultados

9.2 CU-02: Capturar Criatura

Actor Principal: Jugador

Precondiciones:

- En combate con criatura salvaje
- Criatura con vida reducida
- Tener Pokéballs en inventario

Flujo Principal:

1. Jugador selecciona “Capturar”
2. Sistema calcula probabilidad de éxito
3. Mostrar animación de lanzamiento
4. Determinar resultado de captura
5. Actualizar equipo o inventario

9.3 CU-03: Gestionar Equipo

Actor Principal: Jugador

Precondiciones:

- En menú principal o centro Pokémon
- Tener criaturas en equipo o PC

Flujo Principal:

1. Acceder a menú de equipo
2. Ver estadísticas de criaturas
3. Reorganizar orden del equipo
4. Intercambiar entre equipo y PC
5. Guardar cambios automáticamente

10 Modelo y Diseño de la Base de Datos

10.1 Esquema SQLite

```
1 -- Tabla de usuarios
2 CREATE TABLE Users (
3     UserID INTEGER PRIMARY KEY AUTOINCREMENT,
4     Username VARCHAR(50) UNIQUE NOT NULL,
5     PasswordHash VARCHAR(255) NOT NULL,
6     CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP
7 );
8
9 -- Tabla de criaturas base
10 CREATE TABLE Creatures (
11     CreatureID INTEGER PRIMARY KEY,
12     Name VARCHAR(50) NOT NULL,
13     Type1 VARCHAR(20) NOT NULL,
14     Type2 VARCHAR(20),
15     BaseHP INTEGER NOT NULL,
16     BaseAttack INTEGER NOT NULL,
17     BaseDefense INTEGER NOT NULL,
18     BaseSpeed INTEGER NOT NULL,
19     EvolutionLevel INTEGER,
20     EvolvesTo INTEGER
21 );
22
23 -- Tabla de movimientos
24 CREATE TABLE Moves (
25     MoveID INTEGER PRIMARY KEY,
26     Name VARCHAR(50) NOT NULL,
27     Type VARCHAR(20) NOT NULL,
28     Power INTEGER,
29     Accuracy INTEGER,
30     PP INTEGER NOT NULL
31 );
32
33 -- Tabla de partidas guardadas
34 CREATE TABLE SaveGames (
35     SaveID INTEGER PRIMARY KEY AUTOINCREMENT,
36     UserID INTEGER,
37     PlayerData TEXT NOT NULL,
38     CreatedAt DATETIME DEFAULT CURRENT_TIMESTAMP,
39     FOREIGN KEY (UserID) REFERENCES Users(UserID)
40 );
```

10.2 Modelo de Datos en C#

```
1 [System.Serializable]
2 public class Creature
3 {
4     public int CreatureID;
5     public string Name;
6     public int Level;
7     public int CurrentHP;
8     public int MaxHP;
9     public int Attack;
10    public int Defense;
11    public int Speed;
12    public int Experience;
13    public List<Move> Moves;
14    public CreatureType Type1;
15    public CreatureType Type2;
16 }
17
18 [System.Serializable]
19 public class PlayerData
20 {
21     public string PlayerName;
22     public Vector2 Position;
23     public List<Creature> Team;
24     public List<Item> Inventory;
25     public int Money;
26 }
27
28 public enum CreatureType
29 {
30     Normal, Fire, Water, Grass, Electric,
31     Fighting, Poison, Ground, Flying
32 }
```

Listing 2: Clases principales del juego

10.3 Relaciones Principales

- SaveGames.UserID → Users.UserID
- Creatures.EvolvesTo → Creatures.CreatureID
- PlayerData.Team → Creatures.CreatureID

11 Guía de Estilos

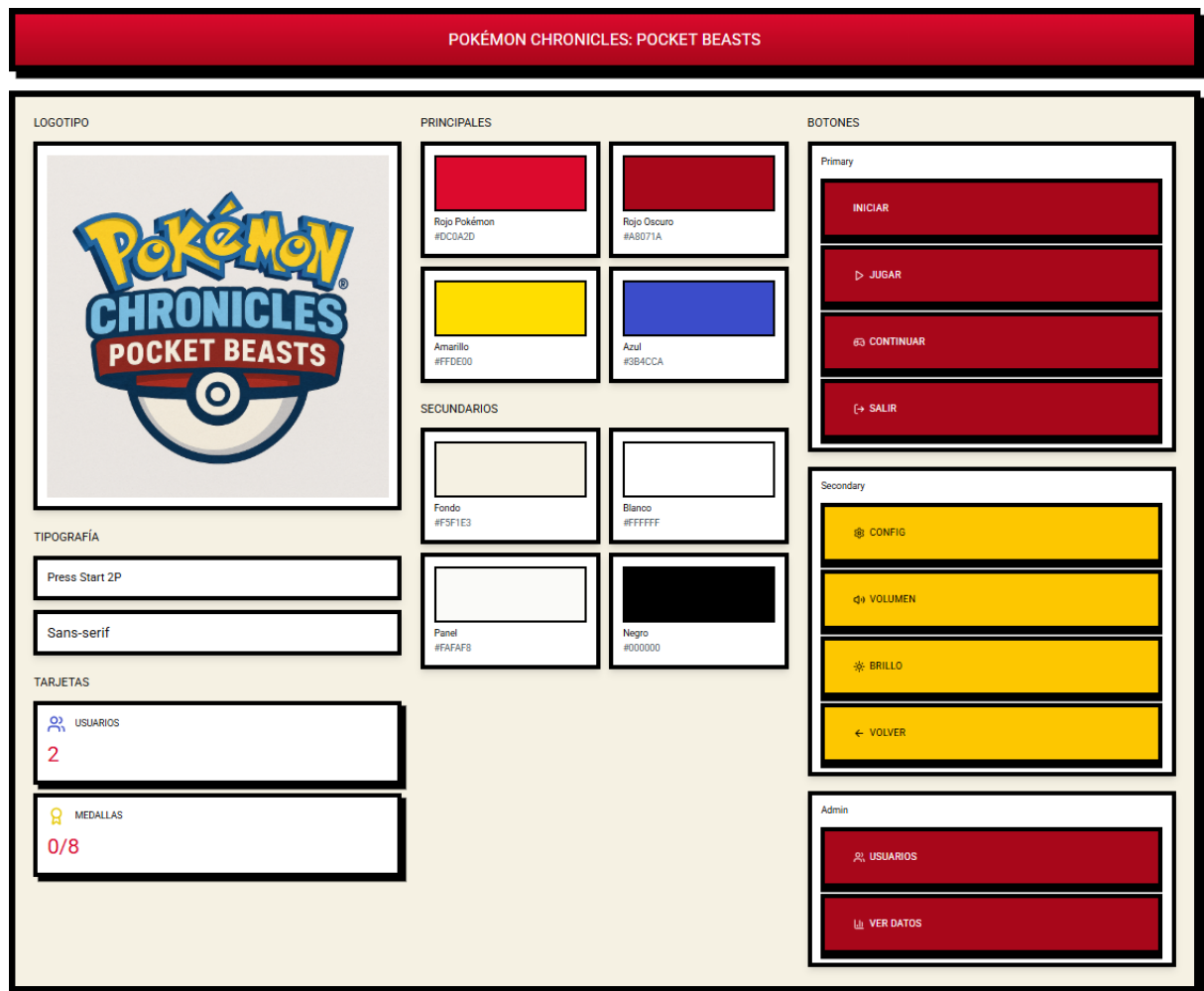


Figura 1: Guía de estilos visual completa del proyecto (colores, tipografías, componentes UI y referencias de diseño).

[Ver guía interactiva en Figma](#)

11.1 Logo

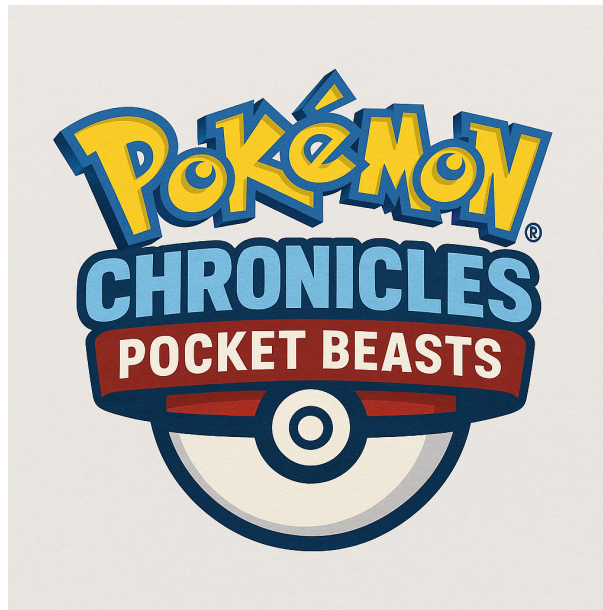


Figura 1.1: Logo principal del juego Pokémon Chronicles: Pocket Beasts.

11.2 Paleta de Colores

Colores Principales

- **Rojo Pokémon (#DC0A2D):** Color primario de los botones de acción principales como “Iniciar”, “Jugar” o “Salir”. Representa energía, pasión y el espíritu competitivo de Pokémon.
- **Rojo oscuro (#A8071A):** Variante más profunda para estados de hover o sombreado, aporta contraste visual.
- **Amarillo (#FFED00):** Usado en botones secundarios y elementos de configuración. Transmite alegría, energía y dinamismo.
- **Azul (#3B4CCA):** Representa la serenidad y equilibrio, usado en botones de administración o secciones informativas.

Colores Secundarios

- **Fondo (#F5F1E3):** Color base del fondo general, proporciona contraste y claridad a los elementos de interfaz.
- **Panel (#FAFAF8):** Paneles y tarjetas de información, aporta ligereza visual.
- **Blanco (#FFFFFF):** Texto o rellenos secundarios para resaltar botones o entradas.
- **Negro (#000000):** Contornos, tipografía y sombras principales.

11.3 Tipografías

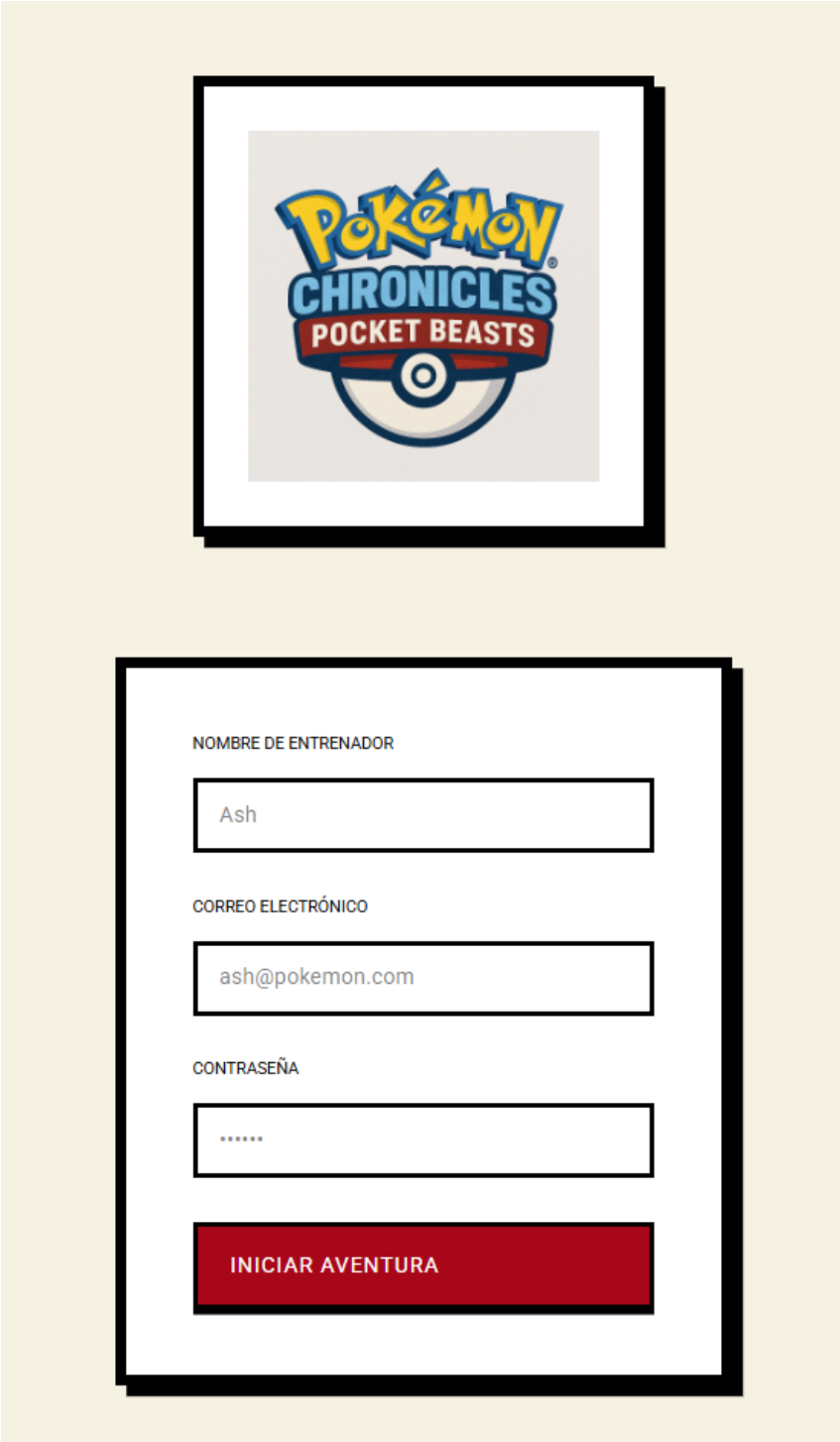
- **Títulos y botones:** Press Start 2P — tipografía pixelada, retro y distintiva, que remite a los juegos clásicos de Pokémon.
- **Texto general:** Sans-serif — utilizada para mantener la legibilidad en pantallas de diferentes tamaños.

11.4 Componentes UI

- **Botones primarios:** Rojos con texto blanco, alto contraste y bordes oscuros.
- **Botones secundarios:** Amarillos con texto negro, para acciones de configuración o navegación secundaria.
- **Campos de texto:** Rectangulares, con bordes negros y fondo blanco para resaltar sobre el fondo beige.
- **Tarjetas informativas:** Contadores de usuarios y medallas en la interfaz del administrador.

12 Mockups de Interfaz

12.1 Pantalla de Login



The mockup shows a login interface for 'Pokémon Chronicles Pocket Beasts'. At the top is the game's logo, which features the word 'Pokémon' in its signature font, 'Chronicles' below it, and 'Pocket Beasts' on a red banner. Below the logo is a login form with three input fields and a button. The first field is labeled 'NOMBRE DE ENTRENADOR' and contains the text 'Ash'. The second field is labeled 'CORREO ELECTRÓNICO' and contains the text 'ash@pokemon.com'. The third field is labeled 'CONTRASEÑA' and contains a series of dots to mask the password. Below these fields is a red button with the text 'INICIAR AVENTURA' in white capital letters.

Figura 2.1: Interfaz de inicio de sesión. El jugador introduce su nombre, correo y contraseña para comenzar la aventura.

12.2 Pantalla de Login del Usuario

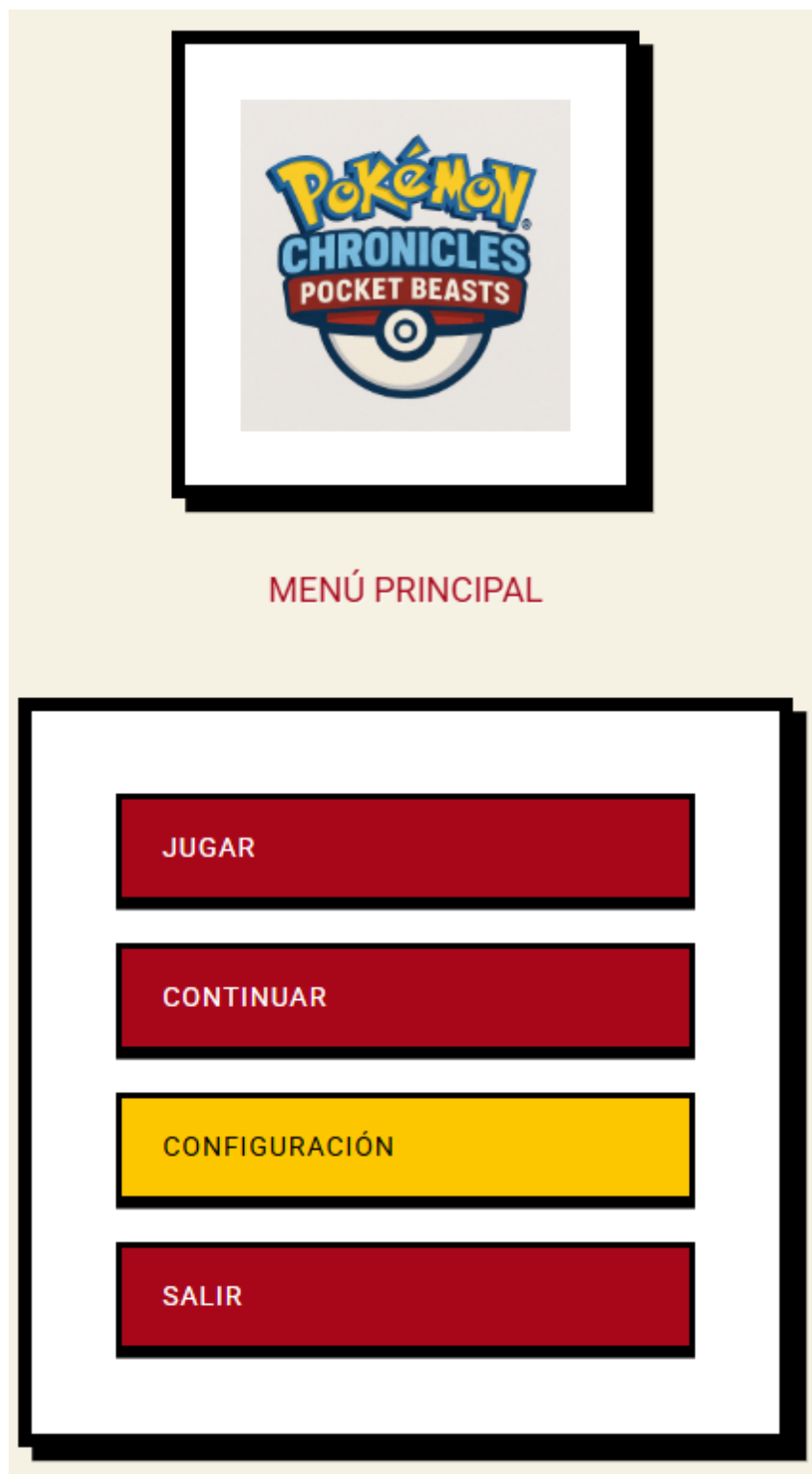


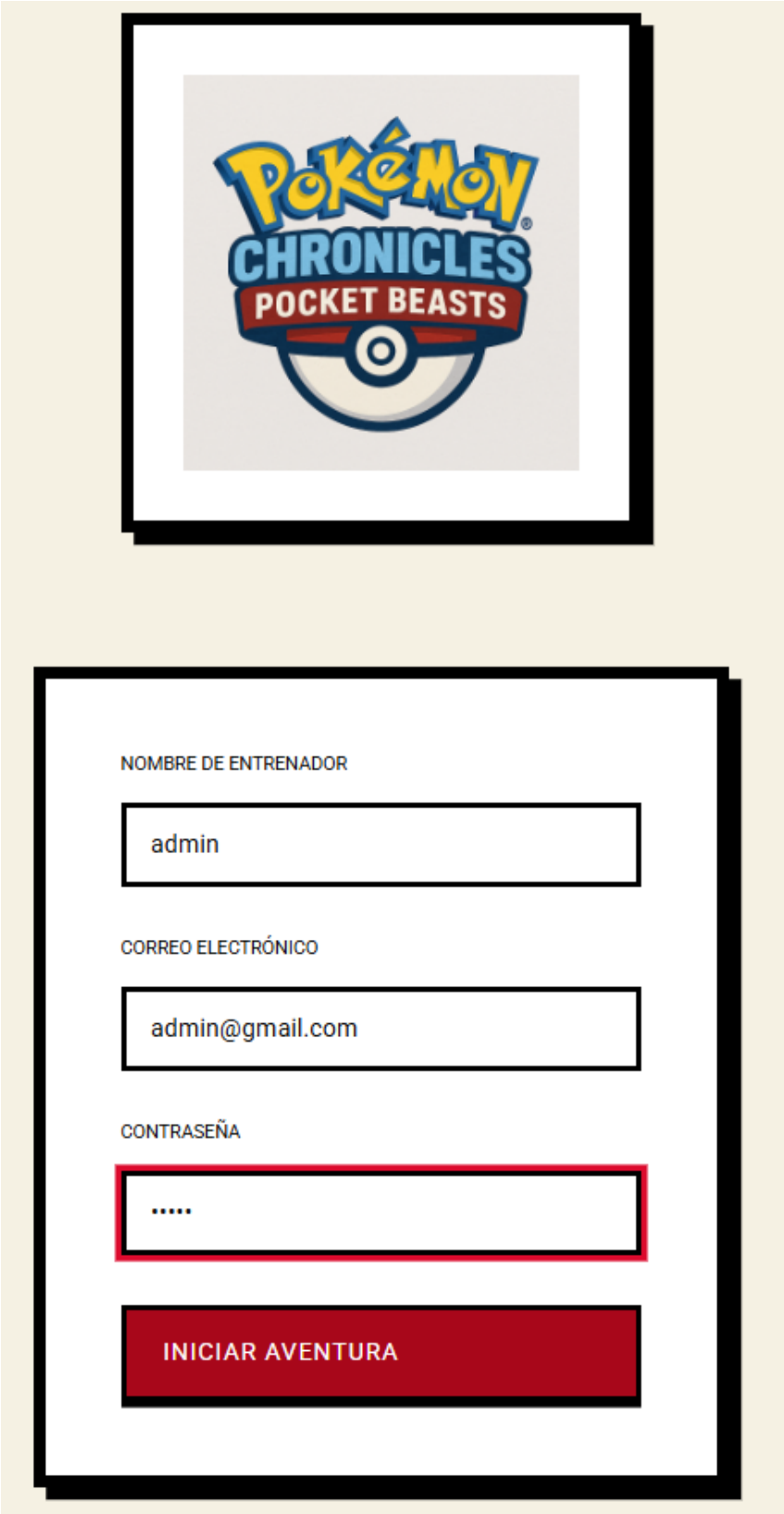
Figura 2.2: Interfaz del login del usuario normal.

12.3 Pantalla de configuración del Usuario



Figura 2.3: Pantalla de configuración del usuario.

12.4 Pantalla de login del admin



Pokémon
CHRONICLES
POCKET BEASTS

NOMBRE DE ENTRENADOR

admin

CORREO ELECTRÓNICO

admin@gmail.com

CONTRASEÑA

.....

INICIAR AVENTURA

Figura 2.4: Interfaz del login del admin. El nombre para entrar sería “admin” y la contraseña “admin”.

12.5 Pantalla del panel del admin

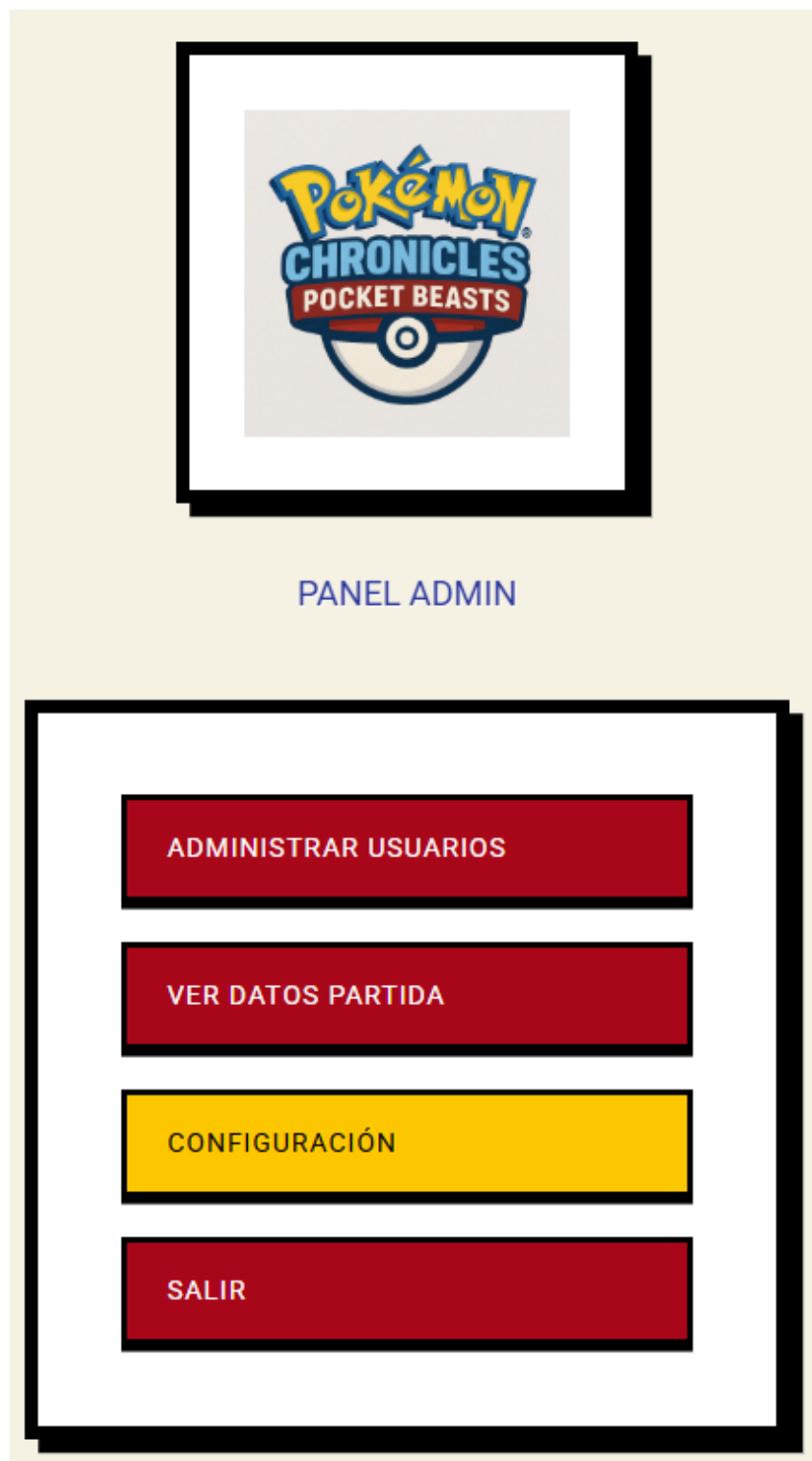


Figura 2.5: Interfaz del panel del admin.

12.6 Pantalla del administrar usuarios

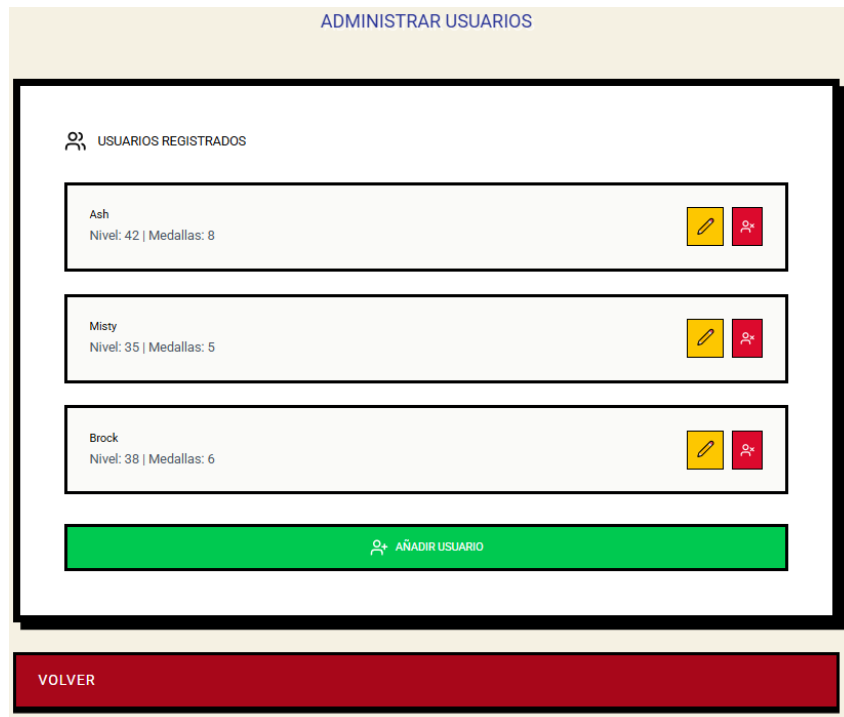


Figura 2.6: Interfaz del administrar usuarios.

12.7 Pantalla de los datos de la partida

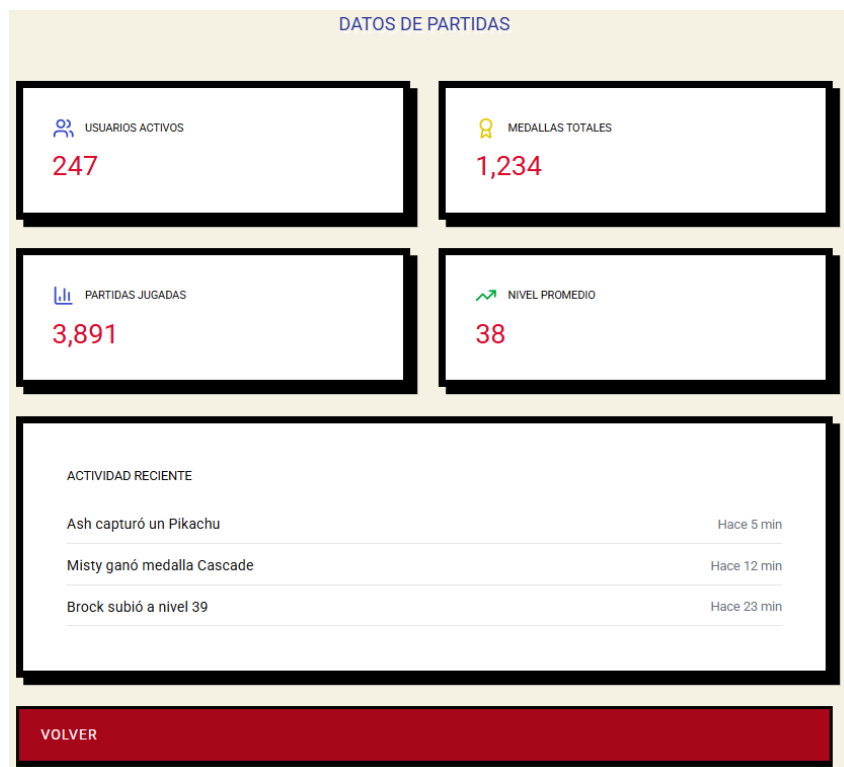


Figura 2.7: Interfaz de los datos de la partida.

12.8 Pantalla de configuración del Admin



Figura 2.8: Interfaz de configuración del admin.

12.9 Pantalla de Flujos

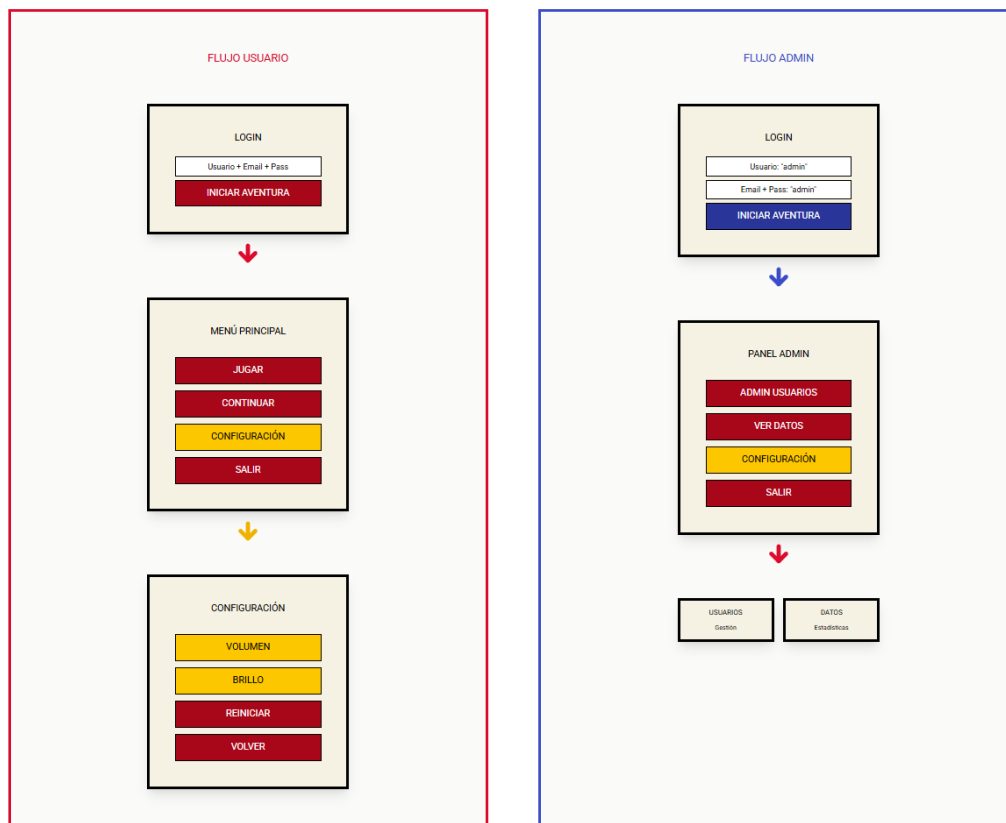


Figura 2.9: Flujos de navegación para usuario y administrador. El usuario accede al menú principal con opciones de juego y configuración, mientras que el administrador gestiona usuarios y datos.

12.10 Pantalla Principal del Usuario

- **Jugar:** Inicia una nueva partida desde el principio.
- **Continuar:** Retoma la partida guardada.
- **Configuración:** Acceso a opciones de volumen, brillo o reinicio.
- **Salir:** Cierra la aplicación.

12.11 Pantalla del Administrador

- **Admin Usuarios:** Permite visualizar, editar o eliminar cuentas.
- **Ver Datos:** Muestra estadísticas globales de progreso y uso.
- **Configuración:** Acceso a parámetros técnicos o ajustes del sistema.
- **Salir:** Finaliza la sesión administrativa.

13 Flujo de Navegación

13.1 Mapa de Navegación

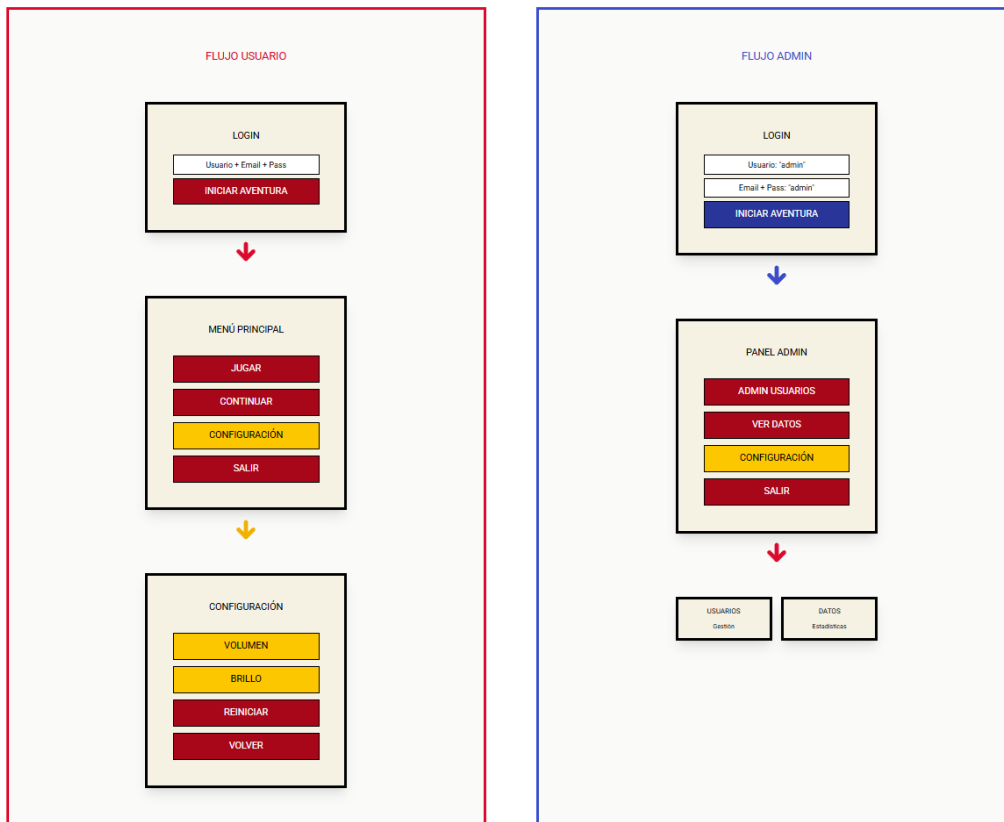


Figura 3.1: Mapa de navegación entre pantallas principales del usuario y del administrador.

13.2 Descripción del Flujo

- Inicio → Login
- Si el usuario inicia sesión con credenciales normales:
 - Accede al menú principal (Jugar, Continuar, Configuración, Salir).
- Si el usuario inicia sesión con credenciales “admin”:
 - Accede al panel de administración (Usuarios, Datos, Configuración, Salir).

14 Manual de Usuario

Este manual describe el funcionamiento actual del sistema desde la perspectiva del usuario final. Incluye las pantallas implementadas, sus funcionalidades y el flujo completo de registro e inicio de sesión.

14.1 Pantalla de Inicio de Sesión

La pantalla de inicio de sesión permite al usuario acceder al sistema introduciendo sus credenciales:

- **Nombre de entrenador:** Identificador único del jugador
- **Correo electrónico:** Dirección asociada a la cuenta
- **Contraseña:** Clave de seguridad personal

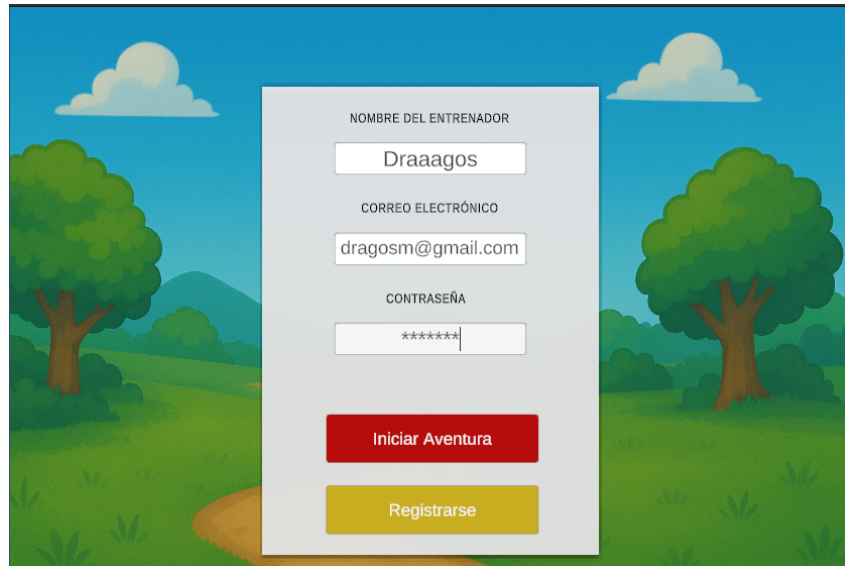
La imagen muestra una interfaz de usuario para iniciar sesión. El fondo es un paisaje con árboles verdes y un cielo azul con nubes. En el centro hay un formulario gris con los siguientes campos: 'NOMBRE DEL ENTRENADOR' con el valor 'Draaagos', 'CORREO ELECTRÓNICO' con el valor 'dragosm@gmail.com', y 'CONTRASEÑA' con el valor '*****'. Debajo de los campos hay dos botones: uno rojo con el texto 'Iniciar Aventura' y uno amarillo con el texto 'Registrarse'.

Figura 11.1: Pantalla de inicio de sesión con credenciales válidas

14.1.1 Mensajes de Error

Cuando las credenciales son incorrectas, el sistema muestra un mensaje de error específico:

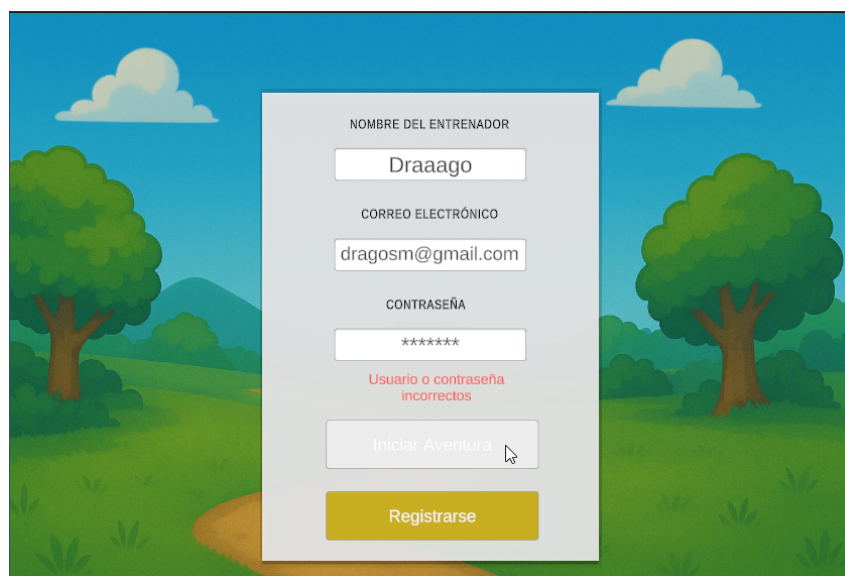
La imagen muestra la misma interfaz de usuario que la Figura 11.1, pero con un mensaje de error. El campo de contraseña ahora muestra '*****' y debajo de él hay un mensaje de error en rojo que dice 'Usuario o contraseña incorrectos'. El botón 'Iniciar Aventura' ahora es gris y tiene un cursor de mouse encima, lo que indica que no es clicable. El botón 'Registrarse' sigue siendo amarillo y clicable.

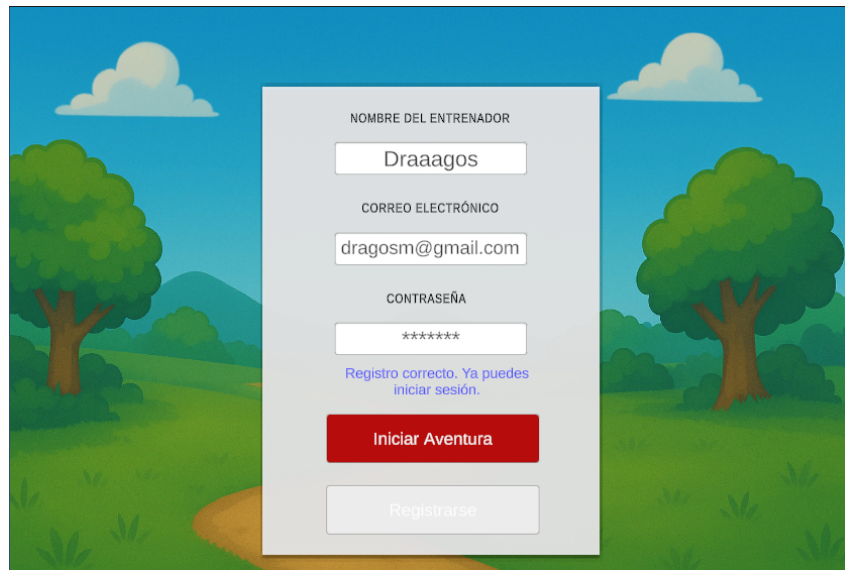
Figura 11.2: Mensaje de error tras intento de login fallido

14.2 Pantalla de Registro de Nuevo Usuario

La pantalla de registro permite crear una nueva cuenta en el sistema con las siguientes validaciones:

- **Formato de correo válido:** Debe seguir la estructura `usuario@dominio.com`

- **Contraseña segura:** Mínimo 6 caracteres
- **Usuario único:** Verificación en tiempo real con el backend



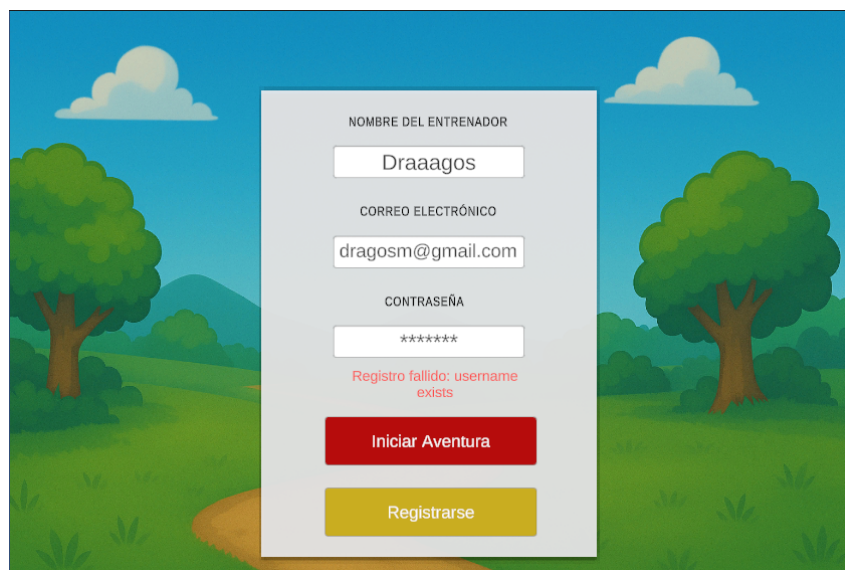
A registration form is displayed against a cartoon background of a green field, two large trees, and a blue sky with clouds. The form has the following fields and elements:

- NOMBRE DEL ENTRENADOR:** A text input field containing "Draaagos".
- CORREO ELECTRÓNICO:** A text input field containing "dragosm@gmail.com".
- CONTRASEÑA:** A password input field containing "*****".
- Feedback message:** "Registro correcto. Ya puedes iniciar sesión." in blue text.
- Buttons:** A red button labeled "Iniciar Aventura" and a grey button labeled "Registrarse".

Figura 11.3: Registro completado exitosamente

14.2.1 Validaciones de Registro

El sistema previene errores comunes durante el registro:



The same registration form as in Figure 11.3 is shown, but with an error message and different button states:

- NOMBRE DEL ENTRENADOR:** "Draaagos"
- CORREO ELECTRÓNICO:** "dragosm@gmail.com"
- CONTRASEÑA:** "*****"
- Error message:** "Registro fallido: username exists" in red text.
- Buttons:** The "Iniciar Aventura" button is red, and the "Registrarse" button is yellow.

Figura 11.4: Error al registrar usuario (nombre existente)

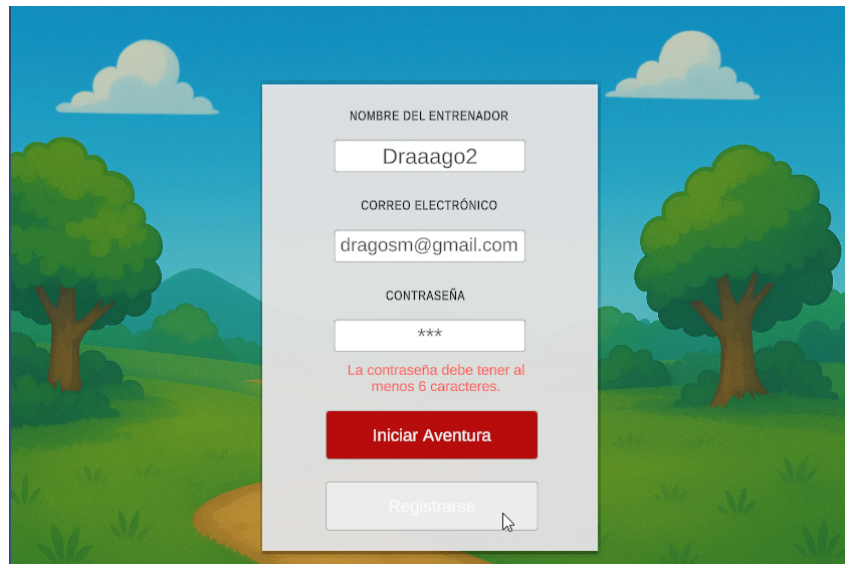


Figura 11.5: Error al registrar usuario (contraseña existente)

14.3 Acceso al Mundo del Juego

Tras un inicio de sesión exitoso, el juego carga automáticamente la escena principal del mundo:



Figura 11.6: Escena del mundo principal tras inicio de sesión exitoso

Esta pantalla incluye:

- Movimiento del personaje principal
- Elementos interactivos del entorno
- Sistema de navegación básico
- Preparación para encuentros con criaturas

15 Pruebas y Validación del Sistema

15.1 Pantallas Implementadas y Estado

La siguiente tabla resume las pantallas actualmente implementadas y su estado de prueba:

Pantalla	Funcionalidad	Estado
Inicio de Sesión	Autenticación de usuarios, validación de credenciales, comunicación con backend	Probada
Registro	Creación de nuevas cuentas, validación de datos, verificación de unicidad	Probada
Mensajes de Error	Feedback visual para errores de formato, conexión y credenciales	Probada
Escena del Juego	Mundo principal con movimiento básico y elementos interactivos	Probada

Cuadro 3: Estado actual de implementación de las pantallas del sistema

15.2 Casos de Prueba Ejecutados

- **Login exitoso:** Usuario válido → Backend devuelve token JWT de autenticación
- **Login fallido:** Contraseña incorrecta → Mensaje de error visual específico
- **Registro correcto:** Datos válidos y usuario nuevo → Cuenta creada exitosamente
- **Registro fallido (usuario repetido):** Nombre existente → Mensaje “username exists”
- **Registro fallido (contraseña corta):** Validación cliente → Advertencia inmediata
- **Error de conexión:** Backend no disponible → Mensaje “Error de conexión”

16 Despliegue y Configuración del Backend

16.1 Entorno Local de Desarrollo

Durante la fase actual de desarrollo, el backend se ejecuta en **entorno local** utilizando:

- **Framework:** Flask (Python)
- **Base de datos:** SQLite
- **Autenticación:** JWT (JSON Web Tokens)
- **Hashing:** bcrypt para contraseñas

16.2 Ejecución del Servidor

El servidor backend se inicia con el siguiente comando:

```
1 python app.py
```

Listing 3: Inicio del servidor Flask local

El servidor expone la API en las siguientes direcciones:

- `http://127.0.0.1:5000` (localhost)

- `http://0.0.0.0:5000` (accesible desde la red local)

```

PS C:\Users\drago> cd D:
PS D:\> cd 'D:\Proyectos\Pokemon TFG\backend\'
PS D:\Proyectos\Pokemon TFG\backend> python .\app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.140:5000
Press CTRL+C to quit
D:\Proyectos\Pokemon TFG\backend\app.py:85: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
or removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime
.UTC).
  "exp": datetime.utcnow() + timedelta(hours=6)
127.0.0.1 - - [05/Dec/2025 17:18:16] "POST /api/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [05/Dec/2025 17:19:07] "POST /api/auth/login HTTP/1.1" 401 -
127.0.0.1 - - [05/Dec/2025 17:19:24] "POST /api/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [05/Dec/2025 21:01:44] "POST /api/auth/login HTTP/1.1" 401 -
PS D:\Proyectos\Pokemon TFG\backend> python .\app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.140:5000
Press CTRL+C to quit
127.0.0.1 - - [05/Dec/2025 21:02:07] "POST /api/auth/register HTTP/1.1" 200 -
D:\Proyectos\Pokemon TFG\backend\app.py:85: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
or removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime
.UTC).
  "exp": datetime.utcnow() + timedelta(hours=6)
127.0.0.1 - - [05/Dec/2025 21:02:09] "POST /api/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [05/Dec/2025 21:02:24] "POST /api/auth/login HTTP/1.1" 401 -
127.0.0.1 - - [05/Dec/2025 21:07:04] "POST /api/auth/login HTTP/1.1" 401 -
127.0.0.1 - - [05/Dec/2025 21:20:11] "POST /api/auth/register HTTP/1.1" 200 -
127.0.0.1 - - [05/Dec/2025 21:22:06] "POST /api/auth/register HTTP/1.1" 200 -
127.0.0.1 - - [05/Dec/2025 21:23:20] "POST /api/auth/login HTTP/1.1" 401 -
127.0.0.1 - - [05/Dec/2025 21:26:07] "POST /api/auth/register HTTP/1.1" 200 -
127.0.0.1 - - [05/Dec/2025 21:26:35] "POST /api/auth/register HTTP/1.1" 400 -
127.0.0.1 - - [05/Dec/2025 21:26:56] "POST /api/auth/login HTTP/1.1" 401 -
127.0.0.1 - - [05/Dec/2025 21:27:51] "POST /api/auth/login HTTP/1.1" 200 -

```

Figura 13.1: Registros del backend mostrando peticiones de login y registro

16.3 Arquitectura del Backend

El backend implementa las siguientes funcionalidades principales:

```

1 import os
2 import sqlite3
3 from flask import Flask, request, jsonify
4 from flask_cors import CORS
5 import bcrypt
6 import jwt
7 from datetime import datetime, timedelta
8
9 app = Flask(__name__)
10 CORS(app)
11
12 # Configuración de seguridad
13 JWT_SECRET = "clave_secreta_jwt_pokemon_chronicles"
14
15 # Configuración de base de datos
16 DATA_DIR = os.path.join(os.getcwd(), "data")
17 DB_PATH = os.path.join(DATA_DIR, "pokemon.db")
18 os.makedirs(DATA_DIR, exist_ok=True)
19
20 # Inicialización de base de datos
21 def init_db():
22     conn = sqlite3.connect(DB_PATH)
23     cur = conn.cursor()
24     cur.execute("""
25     CREATE TABLE IF NOT EXISTS users (
26         id INTEGER PRIMARY KEY AUTOINCREMENT,

```



```

27         username TEXT UNIQUE NOT NULL,
28         password_hash TEXT NOT NULL,
29         email TEXT,
30         created_at DATETIME DEFAULT CURRENT_TIMESTAMP
31     );
32     """
33     conn.commit()
34     conn.close()
35
36 # Endpoints principales
37 @app.route("/api/auth/register", methods=["POST"])
38 def register():
39     # Logica de registro de nuevo usuario
40
41 @app.route("/api/auth/login", methods=["POST"])
42 def login():
43     # Logica de autenticación de usuario
44
45 if __name__ == "__main__":
46     app.run(host="0.0.0.0", port=5000)

```

Listing 4: Estructura principal del backend Flask

16.4 Preparación para Despliegue en Proxmox

Aunque actualmente el backend opera en entorno local, la arquitectura está preparada para despliegue en:

- **Proxmox VE:** Mediante máquina virtual o contenedor LXC
- **Base de datos:** SQLite

16.4.1 Plan de Migración a Producción

1. **Fase 1:** Pruebas exhaustivas en entorno local
2. **Fase 2:** Configuración del contenedor LXC en Proxmox
3. **Fase 3:** Migración de base de datos y configuración
4. **Fase 4:** Pruebas de integración con cliente Unity
5. **Fase 5:** Puesta en producción con URL oficial

16.5 Consideraciones de Seguridad

- **Contraseñas:** Almacenadas con hash bcrypt
- **Tokens:** JWT con expiración de 6 horas
- **CORS:** Configurado para permitir conexiones desde Unity
- **Validación:** Entrada de datos sanitizada en ambos extremos